QALight

Frontend-розробка

Заняття 26. Класи

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Поля та методи класів. Інкапсуляція та модифікатори доступу. Класи та захист типів. Властивості та перевизначення параметрів. Захисні елементи типів. Інтерфейси класів. Створення класу ShoppingCart.

Класи

Поля та методи класів. Інкапсуляція та модифікатори доступу. Класи та захист типів. Властивості та перевизначення параметрів. Захисні елементи типів. Інтерфейси класів.

Класи

Теорія: TypeScript

Класи - це функція JavaScript, що дозволяє інкапсулювати дані та поведінку в єдину сутність. Вони є фундаментальним елементом об'єктно-орієнтованого програмування та використовуються для створення об'єктів, які мають властивості й методи.

Для визначення класу використовується ключове слово `**class**`, а для створення екземплярів цього класу - ключове слово `new`.

TypeScript додає до класів механізм статичної перевірки типів, що допомагає виявляти помилки та забезпечувати дотримання певної структури коду.

Створення класу

Теорія: TypeScript

Найпростіший клас - порожній

```
// Клас Кошика Покупок
class Point {
}
```

Цей клас не дуже корисний, тож почнімо додавати до нього певні члени.

```
class Point {
 x: number;
 //   Property 'x' has no initializer and is not definitely assigned in the constructor.ts(2564)
 // (property) Point.x: number
 y: number;
}
```

Оголошення поля створює в класі **публічну властивість**, доступну для запису.

Створення екземпляру класу

Теорія: TypeScript

Ключове слово `new` створює новий екземпляр класу `Album`, а конструктор встановлює початкові значення властивостей цього класу. У цьому випадку, оскільки значення властивостей задано безпосередньо в коді (hardcoded), кожен екземпляр класу `Album` матиме однакові значення.

```
const pt = new Point();
pt.x = 0;
pt.y = 0;
```

Тепер, коли ми створюємо новий екземпляр класу, ми можемо отримати доступ до властивостей і значень, визначених у конструкторі:

```
console.log(`${pt.x}, ${pt.y}`);
```

Конструктор класу

Теорія: TypeScript

Конструктор - це спеціальний метод, який виконується під час створення нового екземпляра класу. Саме в ньому можна задати початковий стан об'єкта.

Додамо конструктор, який присвоює значення властивостям класу Greeter:

```
class Greeter {
 name: string;
 constructor() {
   this.name = "hello";
 }
}
```

Поле має бути ініціалізоване в конструкторі

Теорія: TypeScript

Поле має бути ініціалізоване безпосередньо в конструкторі. TypeScript не аналізує методи, які ви викликаєте з конструктора, щоб виявити ініціалізацію, оскільки похідний клас може перевизначити ці методи й не виконати ініціалізацію відповідних членів класу.

Якщо ви плануєте ініціалізувати поле в інший спосіб, а не через конструктор (наприклад, якщо зовнішня бібліотека заповнює частину даних вашого класу), ви можете скористатися **оператором твердження про обов'язкове присвоєння — `!`**:

```
class Greeter {
 // Not initialized, but no error
 name!: string;
}
```

Анотація типу

Теорія: TypeScript

Анотація типу є необов'язковою; якщо її не вказати, вона за замовчуванням вважатиметься `**any**`.

Поля також можуть мати **ініціалізатори** - вони виконуватимуться автоматично під час створення екземпляра класу

```
class Point {
 x = 0;
 y = 0;
}
const pt = new Point();
// Prints 0, 0
console.log(`${pt.x}, ${pt.y}`);
```

Так само, як і у випадку з const, let та var, **ініціалізатор властивості класу використовуватиметься для виведення її типу**::

```
const pt = new Point();
pt.x = "0";
// Type 'string' is not assignable to type 'number'.
```

Виведення типів (inference)

Теорія: TypeScript

Не завжди потрібно явно вказувати типи властивостей класу. TypeScript здатний виконувати інтелектуальне виведення типів (**inference**) під час роботи з класами. TypeScript може визначити типи властивостей на основі того, де ми присвоюємо їм значення в конструкторі, тому ми можемо опустити деякі анотації типів:

```
class Album {
 title;
 artist;
 releaseYear;

 constructor() {
   this.title = "Loop Finding Jazz Records";
   this.artist = "Jan Jelinek";
   this.releaseYear = 2001;
 }
}
```

Однак часто можна зустріти визначення типів безпосередньо в тілі класу, оскільки вони слугують своєрідною документацією, яку легко й швидко читати.

Додавання аргументів до конструктора

Теорія: TypeScript

Конструктор можна використовувати для оголошення аргументів класу. Це дозволяє передавати значення під час створення нового екземпляра класу.

Нехай конструктор приймає аргумент `**opts**`, який містить властивості класу `Album`:

В тілі конструктора ми присвоїмо властивостям `this.title`, `this.artist` і `this.releaseYear` значення з аргументу `opts`.

```
// inside the Album class
constructor(opts: { title: string; artist: string; releaseYear: number }) {
 this.title = opts.title;
 this.artist = opts.artist;
 this.releaseYear = opts.releaseYear;
}
```

Ключове слово `this`

Теорія: TypeScript

Ключове слово `this` посилається на екземпляр класу і використовується для доступу до властивостей та методів цього класу.

Коли ми створюємо новий екземпляр класу `Album`, ми можемо передати об'єкт із властивостями, які хочемо встановити.

```
const loopFindingJazzRecords = new Album({
 title: "Loop Finding Jazz Records",
 artist: "Jan Jelinek",
 releaseYear: 2001,
});

console.log(loopFindingJazzRecords.title); // Output: Loop Finding Jazz Records
console.log(loopFindingJazzRecords.artist); // Output: Jan Jelinek
console.log(loopFindingJazzRecords.releaseYear); // Output: 2001
```

Використання класу як типу

Теорія: TypeScript

Цікавою особливістю класів у TypeScript є те, що їх можна використовувати як типи для змінних і параметрів функцій. Синтаксис нагадує використання будь-якого іншого типу чи інтерфейсу.

Використаємо клас `Album` для визначення типу параметра `album` у функції `printAlbumInfo`:

```
function printAlbumInfo(album: Album) {
 console.log(
   `${album.title} by ${album.artist}, released in ${album.releaseYear}.`,
 );
}
```

Після цього ми можемо викликати функцію та передати їй екземпляр класу `Album`:

```
printAlbumInfo(loopFindingJazzRecords);
// Output: Loop Finding Jazz Records by Jan Jelinek, released in 2001.
```

Хоча використання класу як типу є можливим, набагато поширенішим підходом є вимога, щоб класи реалізовували певний інтерфейс.

Властивості класів

Теорія: TypeScript

Ви можете встановлювати значення за замовчуванням для властивостей безпосередньо в тілі класу. Вони називаються **ініціалізаторами властивостей класу**.

```
class Album {
 title = "Unknown Album";
 artist = "Unknown Artist";
 releaseYear = 0;
}
```

Ви можете поєднувати їх із анотаціями типів:

```
class Album {
 title: string = "Unknown Album";
 artist: string = "Unknown Artist";
 releaseYear: number = 0;
}
```

Ініціалізатори властивостей класу

Теорія: TypeScript

Важливо, що ініціалізатори властивостей класу обробляються до виклику конструктора. Це означає, що ви можете змінити значення за замовчуванням, присвоївши інше значення в конструкторі:

```
class User {
 name = "Unknown User";

 constructor() {
   this.name = "Matt Pocock";
 }
}
const user = new User();

console.log(user.name); // Output: Matt Pocock
```

Необов’язкові властивості класу

Теорія: TypeScript

Ми також можемо позначати властивості як необов’язкові так само, як і в об’єктах, використовуючи анотацію `?:`:

```
class Album {
 title?: string;
 artist?: string;
 releaseYear?: number;
}
```

Публічні та приватні властивості

Теорія: TypeScript

Ключові слова `public` і `private` використовуються для керування видимістю та доступністю властивостей класу.
За замовчуванням властивості є публічними (**public**), тобто до них можна отримати доступ ззовні класу.
Якщо ми хочемо обмежити доступ до певних властивостей, ми можемо позначити їх як приватні (private). Це означає, що доступ до них можливий лише зсередини самого класу.
Наприклад, припустімо, ми хочемо додати до класу `album` властивість `rating`, яка використовуватиметься лише всередині цього класу:

```
class Album {
 private rating = 0;
}
```

Якщо тепер ми спробуємо отримати доступ до властивості `rating` з-за меж класу, TypeScript видасть помилку:

```
console.log(loopFindingJazzRecords.rating);
```

Властивість 'rating' є приватною, і доступ до неї можливий лише в межах класу 'Album'.

Публічні та приватні властивості

Теорія: TypeScript

Однак це насправді не перешкоджає доступу до неї під час виконання програми, адже `private` - це лише анотація для етапу компіляції.

Можна ігнорувати цю помилку за допомогою `@ts-ignore` і все одно отримати доступ до властивості:

```
// @ts-ignore
console.log(loopFindingJazzRecords.rating); // Output: 0
```

Приватні властивості під час виконання

Теорія: TypeScript

Щоб отримати таку саму поведінку під час виконання, ви також можете використовувати префікс `#`, щоб позначити властивість як приватну:

```
class Album {
 #rating = 0;
}
```

Синтаксис із символом `#` працює так само, як і модифікатор `private`, але це новіша можливість, що є частиною стандарту ECMAScript. Це означає, що її можна використовувати як у JavaScript, так і в TypeScript.Спроба отримати доступ до властивості, назва якої починається з `#`, з-за меж класу призведе до синтаксичної помилки:

```
console.log(loopFindingJazzRecords.#rating); // SyntaxError
```

Property '#rating' is not accessible outside class 'Album' because it has a private identifier.

Спроба обійти це обмеження шляхом доступу через динамічний рядок поверне `undefined` — і все одно спричинить помилку в TypeScript.

```
console.log(loopFindingJazzRecords["#rating"]); // Output: undefined
```

Отже, якщо ви хочете гарантувати, що властивість є справді приватною, варто використовувати синтаксис із символом `#

Методи класу

Теорія: TypeScript

Окрім властивостей, класи також можуть містити методи. Ці функції визначають поведінку класу та можуть використовуватися для взаємодії як із публічними, так і з приватними властивостями.
Існує кілька способів додавання методів до класу.
Перший із них - діяти за тим самим принципом, що й у випадку з конструктором, і безпосередньо додати метод до тіла класу:

```
// inside of the Album class
printAlbumInfo() {
 console.log(`${this.title} by ${this.artist}, released in ${this.releaseYear}.`);
}
// Another option is to use an arrow function to define the method:
// inside of the Album class
printAlbumInfo = () => {
 console.log(
   `${this.title} by ${this.artist}, released in ${this.releaseYear}.`,
 );
};
loopFindingJazzRecords.printAlbumInfo();
// Output: Loop Finding Jazz Records by Jan Jelinek, released in 2001.
```

Стрілкові функції

Теорія: TypeScript

Стрілкові функції та методи класу відрізняються своєю поведінкою. Ця відмінність полягає в тому, як обробляється ключове слово `this`. Це особливість виконання коду JavaScript:

```
class MyClass {
 location = "Class";
 arrow = () => {  console.log("arrow", this); };
 method() {   console.log("method", this); }
}
const myObj = {
 location: "Object",
 arrow: new MyClass().arrow,
 method: new MyClass().method,
};
myObj.arrow(); // { location: 'Class' }
myObj.method(); // { location: 'Object' }
```

У методі-стрілці (arrow method) `this` прив’язаний до екземпляра класу, в якому цей метод було визначено. У звичайному методі `this` прив’язаний до об’єкта, на якому цей метод було викликано.

Інтерфейси

Теорія: TypeScript

Оголошення інтерфейсу - це ще один спосіб надати ім’я типу об’єкта:

```
/// Інтерфейси (Типи даних)
interface Product {
 id: number;
 name: string;
 category: string;
 price: number;
}
```

Відмінності між псевдонімами типів та інтерфейсами

Теорія: TypeScript

Псевдоніми типів та інтерфейси дуже схожі, і в багатьох випадках можна вільно обирати між ними. Майже всі можливості інтерфейсів доступні й для псевдонімів типів, проте є ключова відмінність: псевдонім типу не можна «відкрити знову» для додавання нових властивостей, тоді як інтерфейс завжди можна розширити.

```
interface Animal {
 name: string;
}
interface Bear extends Animal {
 honey: boolean;
}
const bear = getBear();
bear.name;
bear.honey;
```

Практика

Реалізація кошика покупок

Реалізація кошика покупок

Практика: TypeScript

```
// 1. Інтерфейс для товару
interface Product {
 id: string;
 name: string;
 price: number;
}

// 2. Інтерфейс для елемента у кошику (товар + кількість)
interface CartItem {
 product: Product;
 quantity: number;
}
```

Об’єкт Map для зберігання іменованих колекцій

Практика: TypeScript

Map – це колекція ключ/значення, як і Object, але основна відмінність полягає у тому, що Map дозволяє використовувати ключі будь-якого типу. Тобто ключем в структурі Map може виступати як примітивний тип даних, так і будь-який різновид Object’у.

У Map порядок додавання пар «ключ-значення» зберігається. Якщо ви додали елементи в певному порядку, вони будуть перебиратися в тому ж порядку. Map оптимізований для частих операцій додавання, видалення та пошуку елементів. Ну і звісно Map знає скільки елементів у ньому зберігається, за це відповідає властивість size.

Map створюється за допомогою конструктора new Map() Синтаксис створення Map:

```
const map = new Map();
```

в змінній map буде лежати ось такий об'єкт:

```
Map(0) {
  size: 0 // кількість значень у Map
}
```

Методи та властивості Map

Практика: TypeScript

Методи та властивості:

- new Map() – створює колекцію.
- map.set(key, value) – зберігає значення value за ключем key.
- map.get(key) – повертає значення за ключем; повертає undefined якщо key немає в колекції.
- map.has(key) – повертає true якщо key існує, інакше false.
- map.delete(key) – видаляє елемент (пару ключ/значення) за ключем.
- map.clear() – видаляє всі елементи колекції.
- map.size – повертає поточну кількість елементів.

Для перебору колекції Map є 3 метода:

- map.keys() – повертає об’єкт-ітератор для ключів,
- map.values() – повертає об’єкт-ітератор для значень,
- map.entries() – повертає об’єкт-ітератор зі значеннями виду [ключ, значення], цей варіант типово використовується з for..of.

Використання Map

Практика: TypeScript

Використання Map замість масиву (Array) є чудовим рішенням, коли вам потрібно часто шукати, додавати або видаляти елементи за конкретним ключем. Масиви оптимізовані для впорядкованих списків та доступу за індексами, тоді як Map - це структура типу "ключ-значення".

💡 Коли варто замінити Array на Map?

1. Частий пошук за унікальним ідентифікатором (ID): Якщо у вас є список користувачів або товарів і вам постійно потрібно діставати їх за id.
2. Динамічне додавання та видалення: Коли структура постійно змінюється, і видалення елементів з масиву призводить до падіння продуктивності.
3. Ключем є не просто число: Наприклад, якщо вам потрібно зв'язати дані з конкретним об'єктом (у масиві так зробити не можна).

Клас Кошика Покупок

Практика: TypeScript

```
// 3. Клас Кошика Покупок
class ShoppingCart {
 // Використовуємо Map для швидкого пошуку та оновлення за id товару
 private items: Map<string, CartItem> = new Map();
```

Map<string, CartItem> замість масиву Array дозволяє миттєво знаходити товар за його id, що значно пришвидшує роботу кошика, коли в ньому багато позицій.

Інкапсуляція: стан кошика (items) є приватним (private), тому його не можна випадково зламати ззовні — взаємодія відбувається лише через безпечні методи класу.

Додати товар до кошика

Практика: TypeScript

```
/**
 * Додати товар до кошика. Якщо товар вже є, збільшує його кількість.
 */
public add(product: Product, quantity: number = 1): void {
  if (quantity <= 0) {
    throw new Error("Кількість товару повинна бути більшою за 0");
  }

  if (this.items.has(product.id)) {
    const existingItem = this.items.get(product.id)!;
    existingItem.quantity += quantity;
  } else {
    this.items.set(product.id, { product, quantity });
  }
}
```

Видалити товар з кошика повністю

Практика: TypeScript

```
/**
 * Видалити товар з кошика повністю
 */
public remove(productId: string): boolean {
  return this.items.delete(productId);
}
```

Оновити кількість конкретного товару

Практика: TypeScript

```
/**
 * Оновити кількість конкретного товару
 */
public updateQuantity(productId: string, quantity: number): void {
  if (quantity <= 0) {
    this.remove(productId);
    return;
  }

  const item = this.items.get(productId);
  if (item) {
    item.quantity = quantity;
  } else {
    console.warn(`Товар з id ${productId} не знайдено в кошику`);
  }
}
```

Отримати список усіх елементів у кошику

Практика: TypeScript

```
/**
 * Отримати список усіх елементів у кошику
 */
public getItems(): CartItem[] {
  return Array.from(this.items.values());
}
```

Отримати загальну кількість товарів у кошику

Практика: TypeScript

```
/**
 * Отримати загальну кількість товарів у кошику
 */
public getTotalQuantity(): number {
  let total = 0;
  for (const item of this.items.values()) {
    total += item.quantity;
  }
  return total;
}
```

Порахувати загальну вартість кошика

Практика: TypeScript

```
/**
 * Порахувати загальну вартість кошика
 */
public getTotalPrice(): number {
  let total = 0;
  for (const item of this.items.values()) {
    total += item.product.price * item.quantity;
  }
  // Округлення до 2 знаків після коми для уникнення проблем з розрахунками Float
  return Math.round(total * 100) / 100;
}
```

Математична точність: у методі getTotalPrice додано округлення Math.round(total * 100) / 100, що захищає від відомої помилки JavaScript/TypeScript з дробовими числами на кшталт 0.1 + 0.2 = 0.30000000000000004.

Очистити кошик

Практика: TypeScript

```
 /**
  * Очистити кошик
  */
 public clear(): void {
   this.items.clear();
 }
}
```

Створюємо екземпляр кошика

Практика: TypeScript

```
// Створюємо екземпляр кошика
const cart = new ShoppingCart();

// Наші товари
const laptop: Product = { id: "p1", name: "Ноутбук", price: 25000 };
const mouse: Product = { id: "p2", name: "Мишка", price: 800 };

// 1. Додаємо товари
cart.add(laptop);
cart.add(mouse, 2); // Додали 2 мишки
cart.add(mouse);    // Додали ще одну мишку (разом 3)

console.log("Товари в кошику:", cart.getItems());
console.log("Загальна кількість:", cart.getTotalQuantity()); // Виведе: 4
console.log("Загальна вартість:", cart.getTotalPrice());     // Виведе: 27400
```

Створюємо екземпляр кошика

Практика: TypeScript

```
// 2. Оновлюємо кількість
cart.updateQuantity("p2", 1); // Зменшили кількість мишок до 1
console.log("Нова вартість після оновлення:", cart.getTotalPrice()); // Виведе: 25800

// 3. Видаляємо товар
cart.remove("p1"); // Видалили ноутбук
console.log("Загальна кількість після видалення:", cart.getTotalQuantity()); // Виведе: 1
```

ВПРАВИ

Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Typescriptlang: https://www.typescriptlang.org/

Tsconfig: https://www.typescriptlang.org/tsconfig/

Microsoft for JavaScript developers: https://developer.microsoft.com/en-us/javascript/

JavaScript Guide: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

Nodejs:](https://nodejs.org/uk)

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua